

Feb 18, 2025

Class #1:

Common words related to AI

ChatGPT (By OpenAI), Claude (By Anthropic), Gemini (By Google), DeepSeek (By Deepseek), Llama (By Meta), Mistral (By MistralAI)

 ChatGPT stands for **Chat Generative Pre-Trained Transformer**, which is an LLM (Large Language Model)

Tree

- Transformer Architecture => LSTM (Long Short-Term Memory) => RNN (Recurrent Neural Network) => ANN (Artificial Neural Network)
- AI => Machine Learning => Deep Learning

Related Terms

 **AI** - AI is the simulation of human intelligence in machines. It enables computers to perform tasks that typically require human thought, such as problem-solving, learning, reasoning, and understanding language.

 **Machine Learning** - Machine learning (ML) is a subset of AI that develops algorithms that allow computers to learn from data. Instead of following hard-coded instructions, ML models identify patterns and make decisions independently.

 **Deep Learning** - Deep learning is a branch of machine learning focusing on neural networks with many layers—hence the term “deep.” These layers allow the model to learn highly complex and abstract patterns from large amounts of data.

It is awesome when we have 3 things: -

- Large Datasets
- Complex Pattern
- High Computational Power

 **Neural Network** - A neural network is a computational system inspired by the human brain. It consists of layers of nodes (or "neurons") connected in a structure typically divided into an input layer, hidden layers, and an output layer.

 **Universal Approximation Theorem** - Neural networks can approximate any function if it has enough hidden layers and neurons.

So deep learning is a Universal Function Approximator.

Neural Network Layers

 **Input Layer** - Receives raw data (like pixels of an image or words in a sentence).

 **Hidden Layers** - Process and transform the data. Each node here functions similarly to a simple linear regression model—each one computes a weighted sum of its inputs, applies a threshold (or bias), and passes on the result.

 **Output Layer** - Produces the final result, such as identifying an object in an image or generating a sentence.

Categories of Machine Learning

 **Supervised Learning** - Models are trained on labelled data where the “right answers” are provided.

 **Unsupervised Learning** - Models find patterns or groupings in data without predefined labels.

 **Reinforcement Learning** - Models learn by interacting with an environment and receiving feedback.

Deep-Learning comes under which Category?

It does not come under any category. It is one way of implementing machine learning. We can implement Supervised, Unsupervised or Reinforcement learning using deep learning.

General Steps

- Define Problem Statement
- Collect and Prepare Data
- Split Data
- Define Neural Network Architecture
- Train Neural Network
- Validate Neural Network
- Test Neural Network
- Deploy Neural Network
- Monitor

Usual Data Split - 70% Train, 15% Validation (Optional), 15% Test

- Training data is not used for testing because we want to get unbiased output.
- If we find discrepancies in the test then we will have to make our model better.
- Sometimes validation step is skipped and the split proportion can be adjusted accordingly.

 **Training** - Used to train the model by adjusting its weights and parameters. Typically the largest portion of the dataset.

 **Validation** - The model does not learn from this data but is evaluated to guide improvements. For example after a pass of training the output is validated against this data to see if we are going in the correct direction and whether to continue training or not.

 **Testing** - Used to evaluate the final performance of the trained model. Represents real-world, unseen data to measure generalization ability. No further modifications are made based on test set results.

 **EPOCH** - An epoch is one complete pass through the training dataset by the learning algorithm during the neural network's training process.

Mathematical Concept

 **Linear Regression** - Let's take a simple equation of a line which is $y=mx+c$.

So let's say we want to get the salary from years of experience and we are modelling the relation as a linear regression.

For proof instead of $y=mx+c$, we are temporarily switching the variables to $y=wx+b$.

$y \Rightarrow$ Salary, $\bar{y} \Rightarrow$ Prediction

So the values of w and b are unknown. Now it is going to take random values for these and at the end we are going to calculate how wrong are we. Then we will try to adjust the values of w and b so that the result gets closer to the actual value.

Example

Salary $\Rightarrow$ 20 (value of y), YOE = 5 (value of x)

Equation $\Rightarrow \bar{y} = wx+b$

Random values taken $\Rightarrow w=2, b=5$

Result $\Rightarrow w(5)+2 = 15$

Difference $\Rightarrow y - \bar{y} = 20-15 = 5$

 **Loss/Cost Function** - Measures the error between the predicted and the actual output. It quantifies how far the model's prediction is from the actual value.

$$\text{Loss/Cost} = y - \bar{y}$$

To minimize the error, we are going to implement differentiation in this and to make the concept simpler we are squaring the value so that we can avoid absolute values.

$$\text{Loss/Cost} = (y - \bar{y})^2$$

$$w = w - \alpha(\partial c / \partial w), b = b - \alpha(\partial c / \partial b),$$

α = Learning rate

$$c = (y - \bar{y})^2$$

$$\delta c / \delta w = \delta c / \delta \bar{y} * \delta \bar{y} / \delta w$$

$$\delta c / \delta b = \delta c / \delta \bar{y} * \delta \bar{y} / \delta b$$

$$\delta c / \delta \bar{y} = \delta((y - \bar{y})^2) / \delta \bar{y} = -2(y - \bar{y})$$

$$\delta \bar{y} / \delta w = \delta(wx+b) / \delta w = x$$

$$\delta \bar{y} / \delta b = \delta(wx+b) / \delta b = 1$$

$$\delta c / \delta w = -2(y - \bar{y}) * x = -2x(y - \bar{y})$$

$$\delta c / \delta b = -2(y - \bar{y}) * 1 = -2(y - \bar{y})$$

$w = w - \alpha(-2x(y - \bar{y})) \Rightarrow$ Weight of the connection in the neural network

$b = b - \alpha(-2(y - \bar{y})) \Rightarrow$ Bias value so that the function is generic

These weights and biases are called parameters.

Training Process

- Initialise weights & bias $\Rightarrow$ Random values
- I/P $\Rightarrow$ O/P $\Rightarrow$ Calculate predicted O/P
- Calculate loss/cost $\Rightarrow$ Algorithms
- Compute gradient $\delta c / \delta w, \delta c / \delta b \Rightarrow$ Using chain rule
- Update weights and biases using gradients
- Repeat until loss/cost is minimized

Activation Function

It introduces non-linearity, allowing the network to learn complex patterns.